Session 26 Homework --- Solution Walkthrough

How to use this walkthrough

This document follows the official Session 26 homework exactly, **HW-01 through HW-26**, in the same order and tier structure as the official step sequence. For every step you can compare what you implemented with the official solution: what the step does, the exact file, the exact code added or changed, why, and what capability exists after it. This is the official, published solution -- the code shown here is deliberately complete; this is different from the homework assignment itself, where showing the solution is forbidden.

CORE

HW-01 — CORE: `IssueEditPanel` -- the real priority field

What this step does:

Issue Detail's own priority is now really editable, reusing the identical real "select" field kind LIVE already taught on status.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
            ],
          },
          {
            key: "priority",
            label: "Priority",
            type: "select",
            options: [
              { value: "low", label: "Low" },
              { value: "medium", label: "Medium" },
              { value: "high", label: "High" },
              { value: "urgent", label: "Urgent" },
            ],
          },
        ]}
      />
```

Why: `priority` is a plain real string (`"low"` / `"medium"` / `"high"` / `"urgent"`), the identical real shape as `status` -- the real backend's own `Issue.priority` column is a validated string, never a second Prisma enum concept, so the real "select" field kind already handles it with no change.

After this step: Clean exit. The real assignee field needs one small real panel change first (HW-02/03).

HW-02 — CORE: `IssueEditPanel` -- the real "number" row (type and input)

What this step does:

The `EditableField` type now describes a real `"number"` kind and a real `numeric?` capability on `"select"`, and the real panel can render it -- no real conversion/save yet.

File: `web/src/components/issues/IssueEditPanel.tsx`

Implementation:

TSX

```tsx
// kinds LIVE taught (title, status); HOMEWORK's own priority/assignee fields are mechanical
// transfer of "select," never a new kind. "number" (story points) is the one genuinely new
real
// row kind, needed because a story-points value must reach the backend as a real number,
never
// a string -- `numeric` on a "select" row (assignee) asks for that same real number-or-null
// conversion from an option list, rather than duplicating it as its own fourth kind.
export type EditableField =
  | { key: keyof Issue; label: string; type: "text" }
  | {
      key: keyof Issue;
      label: string;
      type: "select";
      options: { value: string; label: string }[];
      numeric?: boolean;
    }
  | { key: keyof Issue; label: string; type: "number" };
```

TSX

```tsx
      {field.type === "text" || field.type === "number" ? (
        <input
          type={field.type}
          value={value}
```

Why: `numeric` lives on the "select" variant itself (declared at the real call site), not keyed off `field.key === "assigneeId"` inside the component -- the real panel stays entirely driven by its own `fields` prop, never by a hard-coded field name. `type={field.type}` -- `"text"` and `"number"` are both real, valid HTML `<input>` types, so the real field's own `type` string is reused directly, never a second real mapping table.

A real design decision, not an arbitrary order: the type's own `"number"` variant and the input conditional that renders it change together, in this ONE step. Widening the type without widening the renderer (or the reverse) leaves TypeScript unable to narrow `field.options` safely in the `"select"`-only branch, a real compile error, not a style nitpick.

After this step: Clean exit. No real conversion/save happens yet (HW-03).

HW-03 — CORE: `IssueEditPanel` -- the real numeric conversion

What this step does:

The real panel can now save a real number (or `null`) for a field, not only a plain string -- needed before the real assignee and story-points fields (HW-05/06) can work.

File: `web/src/components/issues/IssueEditPanel.tsx`

Implementation:

TSX

```
  async function save(newValue: string) {
    setValue(newValue);
    setPending(true);
    // Real numeric fields (story points; assignee, a nullable foreign key) must reach the
    // backend as a real number or `null`, never the raw string every HTML form control
actually
    // produces -- `@IsInt()` on the real DTO rejects a numeric-looking string outright.
    const numeric = field.type === "number" || (field.type === "select" && field.numeric);
    const payload = numeric ? (newValue === "" ? null : Number(newValue)) : newValue;
    try {
      await updateIssueAction(issue.id, basePath, { [field.key]: payload });
    } finally {
      setPending(false);
    }
  }
```

Why: `newValue === "" ? null : Number(newValue)` -- an emptied real number input must save `null`, never `Number("")` (which is `0`, a real, wrong, valid-looking story-points value).

After this step: Clean exit. Nothing yet sets `numeric: true` (HW-05). `parseInt(newValue)` was deliberately NOT used -- it silently truncates a real non-integer string instead of producing a real, catchable `NaN`.

HW-04 — CORE: Issue Detail -- the real workspace members fetch

What this step does:

The real page now also fetches this issue's own real workspace members -- not rendered yet.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
import type { Issue, Project } from "@/lib/types";
```

TSX

```
import { IssueEditPanel } from "@/components/issues/IssueEditPanel";
```

```
type Member = { userId: number; role: string; user: { id: number; name: string } };
```

TSX

```
  const issue: Issue = await issueRes.json();
  const reporterName = await resolveName(issue.reporterId, token);

  // The real assignee options are this issue's own real workspace's real members -- the
  // identical real membership list Workspace Settings (Session 25) already fetches, reused
here
  // rather than a second, separate "who can be assigned" lookup.
  const projectRes = await apiFetch(`/projects/${projectId}`, { token });
  const project: Project = await projectRes.json();
  const membersRes = await apiFetch(`/workspaces/${project.workspaceId}/members`, { token });
  const members: Member[] = await membersRes.json();
```

TSX

```
          <span>Reporter: {reporterName ?? "unknown"}</span>
```

Why: The real `resolveName(issue.assigneeId, token)` call and its own real display `<span>` are removed together, not kept alongside the new real fetch -- once the assignee becomes a real editable field (HW-05), its own real name comes from the already-fetched `members` list, never a second, redundant lookup or display.

After this step: Clean exit. `members` is not read from anywhere yet (HW-05). Keeping the old `resolveName(issue.assigneeId, token)` call "just in case" would have become real dead code.

HW-05 — CORE: `IssueEditPanel` -- the real assignee field (Reassign Issue)

What this step does:

Issue Detail's own assignee is now really editable -- a real admin/owner (or this issue's own reporter/assignee) can reassign it to any real workspace member, or back to unassigned.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
            ],
          },
          {
            key: "assigneeId",
            label: "Assignee",
            type: "select",
            numeric: true,
            options: [
              { value: "", label: "Unassigned" },
              ...members.map((m) => ({
                value: String(m.userId),
                label: m.user.name,
```

```
              })),
            ],
          },
        ]}
      />
```

Why: The real `"Unassigned"` option has value `""`, not `"0"` or `"null"` -- `save()`'s own real conversion (HW-03) already treats an empty string as `null`, the real way to clear a nullable foreign key. Forgetting `numeric: true` would send the real selected value as the string `"2"`, which the real backend's own `@IsInt()` rejects outright.

After this step: `tsc` / `build` both exit clean; real `2` printed from a direct reassignment request -- the identical real call the panel's own assignee `<select>` makes.

HW-06 — CORE: `IssueEditPanel` -- the real story points field

What this step does:

Issue Detail's own story points are now really editable.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
            ],
          },
          { key: "storyPoints", label: "Story points", type: "number" },
        ]}
      />
```

Why: No `options` at all -- `"number"` is the one real field kind that needs none; the real input itself is the whole real control.

After this step: `tsc` / `build` both exit clean; real `5` printed. `IssueEditPanel` now carries every real field this homework assigns (priority, assignee, story points), on top of LIVE's own title/status.

HW-07 — CORE: `deleteIssueAction`

What this step does:

A real new export exists that deletes an issue against the real backend -- not yet called from anywhere.

File: `web/src/lib/actions/issues.ts`

Implementation:

TSX

```
import { requireSessionToken } from "@/lib/session";

// The real backend's own DELETE /issues/:id already existed before this session -- the real
// homework gap was only ever the frontend's own missing wiring to it, never a new backend
```

```
route.
export async function deleteIssueAction(issueId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/issues/${issueId}`, {
    method: "DELETE",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not delete the issue.");
  }

  revalidatePath(basePath);
  redirect(basePath);
}

export interface IssueFormState {
```

Why: `redirect(basePath)` after a real successful delete -- the real Issue Detail page this request came from no longer has a real issue to show; sending the user back to Project Detail is the only real sensible destination.

After this step: Clean exit. Nothing calls this yet (HW-09). Calling `redirect` before `revalidatePath` would still work here, but the real convention this app's own other actions already use is cache invalidation first, navigation second.

HW-08 — CORE: `DeleteIssueButton` -- the real skeleton

What this step does:

A real button exists, typed for one real issue -- not yet wired to delete anything.

File:
`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/DeleteIssueButton.tsx`

Implementation:

TSX

```
"use client";

import { useState } from "react";

export function DeleteIssueButton({
  issueId,
  basePath,
}: {
  issueId: number;
  basePath: string;
```

```
}) {
  const [pending, setPending] = useState(false);

  return (
    <button
      type="button"
      disabled={pending}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
      Delete issue
    </button>
  );
}
```

Why: `text-red-600` -- the identical real color convention `removeMemberAction`'s own real button (Session 25) already used for a real destructive action.

After this step: Clean exit. The real `onClick` comes next (HW-09); nothing renders this component yet (HW-10).

HW-09 — CORE: `DeleteIssueButton` -- the real confirm and delete

What this step does:

Clicking the real button actually deletes the real issue, after a real confirmation.

File:
`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/DeleteIssueButton.tsx`

Implementation:

TSX

```
import { useState } from "react";
import { deleteIssueAction } from "@/lib/actions/issues";
```

TSX

```
  async function remove() {
    if (!confirm("Delete this issue? This cannot be undone.")) return;
    setPending(true);
    try {
      await deleteIssueAction(issueId, basePath);
    } finally {
      setPending(false);
    }
  }

  return (
    <button
      type="button"
```

```
      disabled={pending}
      onClick={remove}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
```

Why: A real, plain `confirm()` -- this app's own real delete-member flow (Session 25) has no confirmation step at all, but deleting an entire issue is a real heavier, less reversible action, worth one real extra deliberate click. Calling `deleteIssueAction` before the real `confirm()` check would let a real accidental click delete the issue immediately, with no chance to back out.

After this step: Clean exit. Nothing renders this component yet (HW-10).

HW-10 — CORE: wire `DeleteIssueButton` into Issue Detail

What this step does:

Issue Detail now really offers a way to delete the issue.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
import { IssueEditPanel } from "@/components/issues/IssueEditPanel";
import { DeleteIssueButton } from "./DeleteIssueButton";
```

TSX

```
        <div className="mt-4 flex items-center justify-between gap-6 text-sm text-gray-500
dark:text-gray-400">
          <span>Reporter: {reporterName ?? "unknown"}</span>
          <DeleteIssueButton
            issueId={issue.id}
            basePath={`/workspaces/${id}/projects/${projectId}`}
          />
        </div>
```

Why: `items-center justify-between` -- the identical real layout shape `MemberRow` (Session 25) already used to put explanatory text on one side and a real action on the other.

After this step: `tsc` / `build` both exit clean; the real button appears. CORE is complete: every real `IssueEditPanel` field (title, status, priority, assignee, story points) and a real way to delete the issue.

HW-11 — CORE: the real Backlog page -- the real skeleton

What this step does:

A real, visitable page exists that fetches a project's own real data -- no real issues yet.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/backlog/page.tsx`

Implementation:

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";

export default async function BacklogPage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { projectId } = await params;

  const projectRes = await apiFetch(`/projects/${projectId}`);
  if (!projectRes.ok) {
    notFound();
  }
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

Why: The real page re-fetches the real project itself, rather than receiving it as a prop -- the identical real convention every Server Component page in this app already follows.

After this step: Clean exit. The real fetch, filter, and render come next (HW-12).

HW-12 — CORE: the real Backlog page -- the real fetch and filter

What this step does:

The real page now fetches a project's own real issues and filters them down to the real backlog (no sprint).

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/backlog/page.tsx`

Implementation:

TSX

```tsx
import type { Issue, Project } from "@/lib/types";
```

TSX

```tsx
  // The real backend has no sprintId filter param at all -- the identical real situation
  // SortableIssueList (Session 24) already faced for sorting: filtered client-side, over data
  // the backend already returned, never pretending backend support that doesn't exist.
  const issuesRes = await apiFetch(`/issues?projectId=${project.id}`);
  const allIssues: Issue[] = await issuesRes.json();
  const backlog = allIssues.filter((issue) => issue.sprintId === null);

  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
```

```
      <h1 className="text-xl font-semibold">{project.key} — Backlog ({backlog.length})</h1>
    </main>
  );
}
```

Why: `issue.sprintId === null`, never `!issue.sprintId` -- `0` is never a real valid sprint id (ids start at 1), but writing the real exact check this way is still the right habit: a nullable field's own "empty" state should always be compared to `null` explicitly.

After this step: Clean exit. The real back-link comes next (HW-13). Asking the real backend for `?sprintId=null` would be meaningless -- no such real parameter exists.

HW-13 — CORE: the real Backlog page -- back link and description

What this step does:

The real Backlog page now explains itself and offers a real way back to the project.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/backlog/page.tsx`

Implementation:

TSX

```
import Link from "next/link";
import { notFound } from "next/navigation";
```

TSX

```
  const { id, projectId } = await params;
```

TSX

```
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <Link
        href={`/workspaces/${id}/projects/${project.id}`}
        className="text-sm text-blue-600 hover:underline dark:text-blue-400"
      >
        Back to project
      </Link>
      <h1 className="text-xl font-semibold">{project.key} — Backlog ({backlog.length})</h1>
      <p className="text-sm text-gray-500 dark:text-gray-400">
        Every issue not currently in a sprint.
      </p>
    </main>
```

Why: A one-line real explanatory `<p>` -- "Backlog" alone is a real term a first-time learner (and a new teammate) may not yet connect to "no sprint," worth stating plainly once.

After this step: Clean exit. The real issue list itself comes next (HW-14).

HW-14 — CORE: the real Backlog page -- the real issue list; Backlog complete

What this step does:

The real Backlog page now lists every real backlog issue, each linking to its own real Issue Detail page.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/backlog/page.tsx`

Implementation:

TSX

```tsx
import type { Issue, Project } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

TSX

```tsx
      <h1 className="text-xl font-semibold">{project.key} — Backlog</h1>
      <p className="text-sm text-gray-500 dark:text-gray-400">
        Every issue not currently in a sprint.
      </p>
      <div className="flex flex-col gap-3">
        {backlog.map((issue) => (
          <Link
            key={issue.id}
            href={`/workspaces/${id}/projects/${project.id}/issues/${issue.id}`}
          >
            <Card>
              <p className="font-medium">
                {issue.key}: {issue.title}
              </p>
            </Card>
          </Link>
        ))}
        {backlog.length === 0 && (
          <p className="text-sm text-gray-400">Nothing in the backlog.</p>
        )}
      </div>
    </main>
```

Why: The real count moves out of the `<h1>` once the real list itself makes it visually obvious -- showing both would be real, redundant information.

After this step: `tsc` / `build` both exit clean; the real backlog issues appear. Nothing yet links to this page from Project Detail (HW-15).

HW-15 — CORE: link to the real Backlog page; CORE complete

What this step does:

A real signed-in user can actually reach the real Backlog page from Project Detail.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`

Implementation:

TSX

```
          Sprints
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/backlog`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          Backlog
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
```

Why: Placed between "Sprints" and "+ New issue" -- the real link row's own real left-to-right order now matches a student's real planning flow: the board, sprints, the backlog, then creating a new issue.

After this step: `tsc` / `build` both exit clean; the real link appears and navigates to the real Backlog page. HOMEWORK CORE is complete: every real `IssueEditPanel` field (priority, assignee, story points), real issue deletion, and the real Backlog page.

STRETCH

HW-16 — CORE: refresh every issue surface

TS

```
revalidatePath(`${basePath}/backlog`); revalidatePath(`${basePath}/sprints`);
revalidatePath(basePath);
```

HW-17 — CORE: roll back a failed inline edit

TSX

```
catch (e) { setValue(previousValue); setError(e instanceof Error ? e.message : "Could not
update this field."); }
```

HW-18 — CORE: show the inline error

TSX

```
{error && <span className="text-xs text-red-600">{error}</span>}
```

HW-19 — CORE: load Sprint choices

TSX

```
const token = await requireSessionToken();
```

HW-20 — CORE: derive valid Sprint options

TSX

```
const sprintOptions = sprints.filter((sprint) => sprint.status !== "completed");
```

HW-21 — CORE: assign a Backlog issue

TSX

```
<IssueEditPanel issue={issue} basePath={basePath} fields={[{ key: "sprintId", type: "select",
numeric: true, options: sprintOptions }]} />
```

HW-22 — CORE: refresh Sprint-dependent routes

TS

```
revalidatePath(`${basePath}/backlog`); revalidatePath(`${basePath}/board`);
```

HW-23 — CORE: authenticate the Board Sprint read

TSX

```
const token = await requireSessionToken();
```

HW-24 — CORE: render only the active Sprint

TSX

```
const boardIssues = activeSprint ? issues.filter((issue) => issue.sprintId ===
activeSprint.id) : [];
```

HW-25 — STRETCH: `IssueCard` -- the real story-points badge

What this step does:

A real board card shows its own real story points, when set.

File: `web/src/components/board/IssueCard.tsx`

Implementation:

TSX

```
        <span className="text-gray-400">{issue.key}</span> {issue.title}
      </Link>
```

```
      {issue.storyPoints !== null && (
        <span
          title="Story points"
          className="w-fit rounded-full bg-gray-100 px-1.5 py-0.5 text-xs font-medium text-
gray-600 dark:bg-gray-800 dark:text-gray-300"
        >
          {issue.storyPoints}
        </span>
      )}
    </div>
  );
}
```

Why: `issue.storyPoints !== null`, not a truthy check -- a real `0` is a real, meaningful story-point value (a trivial issue), and must still render; only `null` (never estimated) hides the badge.

After this step: `tsc` / `build` both exit clean; a real issue with story points set shows its own real badge on the board. HOMEWORK STRETCH is complete.

CHALLENGE

HW-26 — CHALLENGE: why the Sprint lifecycle is forward-only and server-enforced

What this step does:

The student can explain, in their own words, why a Sprint can only move `planned -> active -> completed` (one step at a time, forward only), why at most one real ACTIVE sprint per project is enforced by a friendly service check and a concurrency-safe database guarantee, and why a real completed sprint's own name/goal can no longer be edited.

Official reasoning / model answer:

A correct answer, in the student's own words, addresses all three of:

1. **Why forward-only, one step at a time.** A real sprint's own lifecycle models a real calendar event that already happened or is happening -- a team does not "un-start" a sprint that is underway, and a sprint that already finished cannot retroactively become active again. Allowing the real status to jump directly from `planned` to `completed`, or backward from `active` to `planned`, would let the real data describe a history that never actually occurred.
2. **Why one real ACTIVE sprint per project uses two enforcement layers.** The service checks for an existing active sprint first so it can return a clear domain-specific `409` message. The PostgreSQL partial unique index on `project_id` where `status = 'active'` is the final integrity boundary: if two concurrent start requests both pass the service check, the database still prevents two active rows. The service improves the user-facing error; the database supplies the concurrency-safe guarantee. Both are useful, and the database owns final integrity.

3. **Why a completed sprint's own name/goal is frozen.** A real completed sprint is a real, closed historical record -- its own real points summary, and any real report built from it later, must describe what the sprint actually was while it ran, not a version quietly edited afterward. Freezing it is the same real principle as never letting a workspace's own Activity Log be edited after the fact.

After this step: CHALLENGE complete: a real, independent synthesis of the Sprint domain's own already-built rules, in writing, with no new product code. HOMEWORK complete.

What the homework asked you to prove

CORE proved the generic `IssueEditPanel` pattern genuinely scales: a new "select" field (priority) needed zero new panel code, a genuinely new "number" kind needed exactly one real type-and-renderer change plus one real conversion function, and every later numeric field (assignee, story points) reused both for free -- then proved the full delete flow and a real client-side-filtered Backlog page both transfer identical real patterns from earlier sessions. STRETCH proved the Board's own existing card can grow a new real read-only field without touching anything else about drag-and-drop. CHALLENGE proved the student can explain, unprompted, why each of the Sprint domain's own three rules exists and is enforced where it is, not just that it exists.

Common mistakes

- Splitting the `EditableField` type growth and the input-rendering widening across two separate steps -- leaves a real, genuine intermediate state where TypeScript cannot narrow `field.options` safely.
- Converting a numeric field with `parseInt(newValue)` instead of `Number(newValue)` -- silently truncates a real non-integer string instead of producing a real, catchable `NaN`.
- Forgetting `numeric: true` on the assignee field -- the real selected value is sent as a string, which the real backend's `@IsInt()` rejects outright.
- Calling `deleteIssueAction` before the real `confirm()` check -- a real accidental click would delete the issue immediately, with no chance to back out.
- Asking the real backend for a `sprintId` query parameter on the Backlog page -- no such real parameter exists; the filter is always client-side.
- Checking `issue.storyPoints` truthily instead of `!== null` -- hides a real, meaningful `0` value.

Exact testing instructions

BASH

```
cd api && PORT=3000 npm run start:dev &
cd web && NEXT_PUBLIC_API_URL=http://localhost:3000 PORT=3001 npm run dev &

# Log in as a seeded user (api/prisma/SEED-AND-LIVE-TEST.md)
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" \
  -d '{"email":"omar.admin@b4fboard.dev","password":"Password123!"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# CORE: priority/assignee/story-points persist
curl -s -X PATCH http://localhost:3000/issues/1 -H "Authorization: Bearer $TOKEN" \
```

```
  -H "Content-Type: application/json" -d '{"priority":"high","assigneeId":2,"storyPoints":5}'
\
  | python3 -c "import
sys,json;d=json.load(sys.stdin);print(d['priority'],d['assigneeId'],d['storyPoints'])"

# CORE: delete
curl -s -X DELETE http://localhost:3000/issues/1 -H "Authorization: Bearer $TOKEN" \
  -o /dev/null -w "%{http_code}\n"

# CORE: Backlog
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/backlog \
  | grep -o "Backlog"

# STRETCH: story-points badge on the board
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/board \
  | grep -o "Story points"
```

To deliberately break something and restore it: remove `numeric: true` from the assignee field entry in `IssueEditPanel`'s own call site -- reassigning an issue now fails with a real backend `400` (the selected value arrives as a string, rejected by `@IsInt()`). Restore `numeric: true` to fix it.

What you should understand after studying

- Why a generic, `fields`-driven panel means every new editable field costs one array entry, never a new component.
- Why the `EditableField` type's own growth and the renderer that reads it must change together, not across separate steps.
- Why a Server Action called directly (`deleteIssueAction` via a button's `onClick`) needs no `useActionState`, but still needs a real confirm step for a destructive action.
- Why the Backlog page's own filter lives entirely client-side, over data the backend already returns -- the identical real reasoning `SortableIssueList` (Session 24) already established.
- Why the Sprint domain's own rules (forward-only, one-active-per-project, frozen-on-completion) are each independently server-enforced, never assumed from the UI.